PROGRAMMING & CS FUNDAMENTALS

Regular Expressions Visual Guide

Pattern matching engines —
NFA vs DFA, backtracking, and the traps.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

What a Regex Engine Does

Compile the pattern once, then simulate a machine over the text.

```
text: "aaabbbbc"  vs  pattern a+ b* c
```

- Two phases: COMPILE (pattern -> machine, once) then MATCH (run over text).
- Compile outside loops: re.compile() once, match many — don't recompile per call.
- Regex flavors differ (PCRE, RE2, .NET) — test on YOUR engine, not a website's.

STEP 2/6

NFA vs DFA Engines

Backtracking search versus precomputed determinism.

NFA (backtracking)

Python, JS, Java■features galore

DFA (RE2/Go)

linear time■guaranteed

```
# NFA: tries paths, backtracks on failure — supports backrefs, lookaround
# DFA: one pass, no backtracking — but NO backreferences possible
# tradeoff: expressive power vs guaranteed linear time
```

- NFA simulation explores alternatives depth-first — worst case exponential (next steps).
- DFA builds all state combos upfront — match time O(n) always, compile can blow up.
- Backreferences (\1) and lookaround NEED backtracking — that's why most engines are NFA.
- RE2/Go chose safety: linear-time guarantee, at the cost of those features.

STEP 3/6

Backtracking, Visualized

Greedy match, fail, give back one, retry — the NFA's walk.

```
pattern: a+ b     text: "aaab"

1. a+ grabs 'aaa' (greedy!)   ->  b vs end: FAIL

2. backtrack: a+ gives back one ->  b vs 'b': MATCH
```

match: 'aaab' — the engine tried the WIDEST grab first

```
  greedy  a+   : grab max, backtrack as needed
  lazy    a+?  : grab min, expand as needed     # opposite strategy
  # 'a+' on 'aaa': greedy tries 3,2,1; lazy tries 1,2,3
```

Greedy vs lazy only changes SEARCH ORDER, not what's matchable. Both find a match if one exists — they differ in WHICH match and how fast.

STEP 4/6

Catastrophic Backtracking

Nested quantifiers: the regex that hangs your server.

```
pattern: (a+)+b      text: 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaac'
# a+ can split 30 a's in 2^30 ways; each fails on final 'c'...
# result: BILLIONS of paths before giving up. DoS via regex!
```

2^30 paths ~ 1 billion dead ends — the engine tries them ALL

- Smell: quantifier INSIDE a quantified group — (a+)+, (a*)*, (a|a)+.
- Overlapping alternatives multiply paths: (a|aa)+ on 'aaaa...' is exponential.
- Never run user-supplied regex on user-supplied text without timeouts (or RE2).
- Fix: rewrite unambiguously — 'a+b' instead of '(a+)+b'; or use possessive/atomic (next).

STEP 5/6

Possessive Quantifiers & Atomic Groups

Grab and NEVER give back — pruning the search tree by fiat.

```
a++b        # possessive: a+ grabs all, NO backtracking into it
(?>a+)b    # atomic group: same idea, scoped
# on 'aaab': a++ takes 'aaa', b matches -> MATCH, zero backtrack steps
# on 'aaac': a++ takes 'aaa', b fails -> FAIL FAST (no 2^n retry)
```

- Use when you KNOW the grabbed text can't participate in later matching.
- Fail-fast > exponential retry: possessive turns ReDoS patterns into instant no-match.
- Not in Python re / JS — available in PCRE, Java, .NET, Onigmo (use the atomic-group spelling).
- Alternative: write the pattern unambiguously so backtracking has nothing to explore.

STEP 6/6

Regex Wisdom

Power tool, sharp edges.

- Readability: use named capture groups, comment patterns with re.X, split monster patterns.
- Validate with tests, not eyeballs: regexes are write-only without a test suite.
- Prefer string methods for simple jobs: 'in', startswith, split beat regex on clarity AND speed.
- Anchors (^ $): unanchored patterns scan every position — anchor when you mean full-match.
- Dot-all and multiline change what . and ^ mean — set flags deliberately.
- Two problems joke is real: if the regex needs a regex to explain it, use a parser.

STEP ✓I✓

Key Takeaways

Regular Expressions — Remember

- Engines compile patterns to machines (NFA or DFA), then run them over text.
- NFA backtracks depth-first (expressive: backrefs, lookaround); DFA runs linear-time (RE2/Go).
- Greedy grabs max then backtracks; lazy grabs min then expands — search order, not power.
- Nested quantifiers ((a+)+) cause catastrophic backtracking — exponential ReDoS; rewrite or go possessive.
- Possessive/atomic groups forbid backtracking into the grab — fail fast instead of hanging.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev